

Business App Deployments: Classification Systems



PURPOSE

This document defines a tiered classification system for AI-assisted ("vibe-coded") internal business applications. The system balances speed of delivery with appropriate governance, enabling teams to move fast on low-risk tooling while ensuring proper oversight as applications grow in scope and criticality.

No Green or Yellow application may be customer-facing under any circumstances. All Green and Yellow business app infrastructure is designed to be locked down to internal networks and authenticated internal users only. In rare cases, a Red application may include customer-facing components, but this is a significant exception requiring explicit approval, additional security review, and adherence to all customer-facing compliance standards.

Classification Summary

Attribute	Green	Yellow	Red
Audience	Single team	Multiple teams	Cross-organizational
Governance	Self-service	Architecture review + PR-based	AI Biz Apps team managed
Deployment	Self-deploy to sandbox	EA team deploys via GitHub workflow	Dedicated CI/CD pipeline
Data Access	Read-only (user-context SSO via ZScaler)	Read/Write to approved systems	Full integration surface
Persistence	Web storage only (persisted, not durable)	Durable with SLA expectations	Production-grade SLA
User Management	None (inherits SSO identity)	ReadOnly / ReadWrite / Admin groups	Custom RBAC per application
Blast Radius	Minimal	Moderate (won't take down a department)	High (business-critical path)
Customer-Facing	Never	Never	Internal only (rare exceptions with explicit approval)

Green: Team-Level Utility Apps

What They Are

Green apps are lightweight, team-scoped tools: dashboards, data views, quick prototypes, and quality-of-life utilities. They exist to let teams move fast without waiting on formal development cycles. Think of them as the modern replacement for the spreadsheet a team passes around, but with real-time data access and a proper interface. They should be thought of as persisted but not durable: they will be available day-to-day, but there is no recovery SLA if something breaks.

Technical Boundaries

- **Read-only data access.** Green apps may query various internal systems using the authenticated user's context (SSO via ZScaler). They may not create, update, or delete records in any system of record. The app never sees data the user would not already have permission to view.
- **No server-side file storage.** There is no server-side or cloud-hosted file storage provisioned for Green apps. Any file output is delivered to the user's browser or local machine.
- **Web storage only for state.** User preferences, settings, and profiles must be stored via browser-native mechanisms (localStorage, sessionStorage, IndexedDB). There is no backend persistence layer.
- **Can write files locally.** Users may export or download files (reports, CSVs, local artifacts), but the application does not write to shared file systems, object stores, or enterprise systems.

Governance and Process

- Teams self-deploy Green apps through the self-service dashboard to a sandboxed development environment.
- No pull request or architecture review is required.
- The creating team is fully responsible for their own app. There is no centralized support. If a Green app breaks, the team fixes it or it stays broken.
- Green apps should include a clear indication of who built them and how to reach that team.
- Teams are welcome to request guidance from Enterprise Architecture at any time. EA can advise on design patterns, integration approaches, data access, and whether an app is approaching Yellow territory.

When Green Is Appropriate

- The app serves one team only.
- It reads data but does not modify anything outside the browser.
- If it disappeared tomorrow, nobody outside the team would notice.
- It does not process sensitive data beyond what the user can already access in source systems.

Example Use Cases

- A reporting dashboard that pulls from a data warehouse and displays team KPIs
- A lookup tool that queries an internal API to surface account or ticket details
- A prototype for a workflow idea the team wants to validate before investing further
- A personal productivity tool that aggregates information from several internal sources

Yellow: Shared or System-Writing Apps

What They Are

Yellow apps go beyond single-team utility. They may serve multiple teams, write data back to certain systems, or handle workflows that affect people outside the originating team. The added capability comes with added process. A Yellow app going down might disrupt workflows for a few teams for a short period but will not take down a department for an extended duration.

Technical Boundaries

- **Read and write access to approved systems.** Yellow apps may integrate with internal APIs, databases, and services with appropriate credentials and scoping. Each write target must be explicitly identified during the architecture review. Blanket write access is not granted.
- **Persistent and durable storage is permitted.** Yellow apps may use databases, caches, and file storage as needed for their function.
- **Shared infrastructure with isolation.** Yellow apps run on shared internal infrastructure but must not interfere with other applications.
- **Simplified access control model.** Three roles are supported:
 - **ReadOnly:** Can view data and outputs but cannot modify anything. Default for most users.
 - **ReadWrite:** Can trigger writes, submit data, and perform standard operations within the app's domain.
 - **Admin:** Can manage users, configuration, and app-level settings.

Access group membership is managed through existing identity and access management tooling. The app owner is responsible for defining which users or groups belong to each tier.

Governance and Process

- Yellow apps must reside in a GitHub repository.
- All changes go through pull requests reviewed by a member of the Enterprise Architecture team.
- Repositories follow the standard GitHub review workflow (branch protection, required reviewers, status checks).
- Deployments are performed by the Enterprise Architecture team.
- The owning team(s) must maintain a basic runbook covering what the app does, who uses it, what systems it writes to, and how to disable it in an emergency.
- The app owner is the primary point of contact for the Enterprise Architecture reviewer and is responsible for documenting write targets and communicating changes to affected teams.

When Yellow Is Appropriate

- The app is shared between two or more teams.
- The app writes to any system (database, API, queue, file store).
- Downtime would cause noticeable disruption beyond a single team.
- The app processes or transforms data that others depend on.

Example Use Cases

- A request intake tool that writes submissions into a ticketing or workflow system
- A shared operational dashboard with editable fields that update a system of record
- A coordination tool used by two or three teams to manage a joint process

Red: Mission-Critical Business Apps

What They Are

Red apps are high-visibility, high-value applications that operate across multiple teams and sit on or near critical business paths. They are the apps where downtime, data errors, or unexpected behavior would be immediately felt across the organization. Red apps are maintained by the AI Biz Apps team, which partners with app owners to manage backlogs, feature development, and operational health.

Technical Boundaries

- **Full integration surface.** Red apps may read from and write to any system approved during architecture review. Data flows are documented, monitored, and subject to change management.
- **Production-grade infrastructure.** Red apps run on dedicated or reserved infrastructure with monitoring, alerting, backup, and disaster recovery appropriate to their criticality.
- **Robust access control.** Role-based access is defined per application based on business requirements, potentially more granular than the Yellow three-role model. Access design is part of the initial architecture review and is revisited as the application evolves.
- **Observability is required.** Logging, metrics, and tracing must be in place. The AI Biz Apps team must be able to diagnose issues without relying solely on the original builder.

Governance and Process

- The AI Biz Apps team owns the codebase, deployment pipeline, and operational responsibility.
- The AI Biz Apps team works with designated app owners (from the business side) to prioritize backlog and plan features. App owners are partners, not passive requestors. They are expected to maintain a prioritized backlog, participate in regular planning sessions, and be available for requirements clarification and user acceptance testing.

- Standard software development lifecycle applies: version control, code review, automated testing, staged deployments.
- Repositories follow the standard GitHub review workflow (branch protection, required reviewers, status checks).
- Incident response follows established organizational protocols.
- For mission-critical applications that outgrow the shared AI Biz Apps team's capacity, the app may graduate to a dedicated development team. This decision is made jointly by AI Biz Apps team leadership and business stakeholders based on factors such as user base size, revenue impact, regulatory exposure, and operational criticality.

When Red Is Appropriate

- The app is used across many teams or an entire business unit.
- Downtime or data errors would cause significant business impact.
- The app requires ongoing feature development, not just maintenance.
- Stakeholders expect production-grade reliability and a support model.
- Someone is asking for an SLA.

Example Use Cases

- A cross-functional planning tool used by operations, finance, and product teams to manage quarterly targets
- A data pipeline management application that coordinates inputs and outputs across several departments
- An internal analytics platform that drives executive reporting and strategic decisions
- A workflow orchestration tool that automates multi-step business processes with financial or compliance implications

Security and Compliance

Regardless of classification, all applications in this system share baseline security properties:

- **Internal network only.** All apps are deployed behind internal network boundaries. No public internet exposure.
- **Authenticated users only.** All access requires SSO authentication via ZScaler.
- **No customer-facing exposure for Green or Yellow.** Green and Yellow apps are strictly internal tools with no external access path.
- **Red apps with customer-facing components are exceptional.** A Red app may include customer-facing elements only with explicit approval from Enterprise Architecture and compliance stakeholders. Such exceptions require additional security hardening, penetration testing, and ongoing compliance review beyond standard Red-tier requirements.
- **Principle of least privilege.** Each app receives only the access it needs for its classification level.

Additional security requirements scale with classification:

Requirement	Green	Yellow	Red
SSO Authentication	Yes	Yes	Yes
Network restriction	Internal only	Internal only	Internal only
Access control groups	N/A (user's access)	ReadOnly / ReadWrite / Admin	Custom RBAC
Code review	None required	Enterprise Architecture	AI Biz Apps team
Vulnerability scanning	Managed	Continuous	Continuous
Snyk Scanning	Managed	Required	Required
Audit logging	None	Recommended	Required
Data classification review	None	On onboarding	On onboarding + periodic
Runbook	None	Required	Required
Observability	None	None	Required
Incident response plan	None	None	Required

Decision Guide

When classifying a new application, work through these questions in order:

1. **Will the app write to any system of record?** If no, it may qualify as Green. If yes, it is at minimum Yellow.
2. **Who uses it?** If only your team, and no writes are needed, Green. If multiple teams, start at Yellow.
3. **What happens if this app goes down for a full business day?** If the answer is "the team finds a workaround," Green or Yellow. If it involves escalation, revenue impact, or compliance risk, Red.
4. **Does the app touch sensitive, regulated, or financial data with write access?** If yes, lean toward Red regardless of team scope.
5. **Does it need to exist long-term with reliability expectations?** If it is disposable or experimental, Green. If it needs to be reliable for months or years, Yellow or Red.
6. **Is someone asking for an SLA?** If yes, it is Red.
7. **Could this app ever need to be customer-facing?** If there is any possibility, stop. This framework does not cover customer-facing applications outside the Red exception process. Engage your product and engineering teams through standard channels.

Lifecycle and Graduation

Applications naturally evolve. A dashboard built for one team (Green) may prove valuable to neighboring teams (Yellow) and eventually become a critical business tool (Red). The classification system supports this growth:

Transition	Trigger	Action Required
Green to Yellow	App needs write access, or adoption spreads beyond one team	Move code to GitHub repo; engage Enterprise Architecture for review; implement access controls; create runbook
Yellow to Red	App becomes high-visibility or business-critical; multiple teams depend on it daily	Transfer ownership to AI Biz Apps team; establish formal app owner relationship; implement production-grade observability, SLAs, and incident response
Red to Dedicated Team	App demands exceed AI Biz Apps team capacity or strategic importance warrants full-time focus	Establish dedicated team; transfer knowledge and ownership

Demotion is also possible. A Red app that loses its user base may be reclassified to Yellow or retired. A Yellow app that only one team uses may return to Green (with write access removed or scoped accordingly).

Responsibilities Summary

Role	Green	Yellow	Red
Building team	Builds, deploys, maintains	Builds, submits PRs, maintains logic and runbook	Provides requirements, manages backlog with AI Biz Apps
Enterprise Architecture	Available for guidance on request	Reviews PRs, deploys, advises on architecture	Consulted on architecture decisions and onboarding
AI Biz Apps team	N/A	N/A	Owns code, deploys, operates, develops features
App Owner (business)	Team owns informally	Defines access groups, documents write targets	Formal role: prioritizes backlog, accepts features, validates outcomes
Users	Self-service access via SSO	Access via assigned group (RO/RW/Admin)	Access via application-specific roles